BIS Compliance Engine

an AI search engine that takes a product description in plain English and returns the top 5 applicable Indian Standards (IS codes) in under a second.

Why it matters:

63 million Indian Micro & Small Enterprises (MSEs) currently spend **3+ weeks** and **₹15,000+** identifying which BIS standards apply to their product. Our tool does it in **0.83 seconds** for free.

Source data:

SP 21 (2005) — a **929-page PDF** from BIS containing **557 building-material standards** (Cement, Steel, Concrete, Aggregates).

Ramesh runs a cement plant in Rajasthan.

He spent 3 weeks and ₹15,000 last year just figuring out which BIS standards apply to his cement. Today, our AI does it in 0.8 seconds. Free.

BIS × SS Hackathon 2026 · RAG Track

63 million Rameshes.

63M

MSEs in India

Small and medium enterprises needing BIS compliance

3 weeks

Average time

To identify applicable standards

₹15,000

Consultant fee

Typical cost per product compliance check

Compliance is the bottleneck nobody talks about. Until now.

Type your product. Get your standards. Done.

Input Query

> "33 grade ordinary Portland cement for general construction"

Results (⏱️ 0.8 seconds)

IS 269: 1989

Ordinary Portland Cement, 33 Grade

94% confidence

IS 4031: 1996

Methods of Physical Tests for Hydraulic Cement

81% confidence

IS 1489 (Part 1): 1991

Portland Pozzolana Cement, Fly Ash Based

62% confidence

Every result is verifiable. Every code is real. Zero hallucinations.

Hybrid retrieval. Cross-encoder reranking. Whitelist filtering.

1. **Product Query**
2. **BM25 + Vector Search**
   Exact matches + semantic intent
3. **Cross-Encoder Reranker**
   Query-document scoring
4. **Whitelist Filter**
   777 valid codes only
5. **Top 5 Results**

HYBRID RETRIEVAL

BM25 catches exact code matches. Vector catches semantic intent. Together they catch everything.

CROSS-ENCODER RERANK

Scores actual query-document pairs. Far more accurate than cosine similarity alone.

WHITELIST FILTER

Extracted all 777 valid IS codes from the PDF at index time. Hallucination is structurally impossible.

Six engineering decisions, measured against naive baselines.

What naive teams do	What we built
500-char chunks (text bleeds across standards)	**Standard-boundary chunking** — split on "SUMMARY OF" marker, 557 atomic chunks
Single vector search	**BM25 + Vector + Cross-encoder** hybrid pipeline
Trust the LLM output	**777-code whitelist** — impossible to return fake codes
No query enhancement	**Domain synonym dict** — cement → portland, opc, hydraulic (0ms)
Cold-start on every query	**Pre-warmed models** — 40s startup → 0.8s queries
Confuse "Part 1" and "Part 2"	**Part-aware parsing** — each part is a separate chunk

Measured. Not claimed.

100%

HIT RATE @3

Target: >80% · **+20pp above**

90%

MRR @5

Target: >70% · **+28% above**

0.83s

Average Latency

Target: <5.0s · **6× faster**

- Tested on the official 10-query public test set
- Verified by the organizer's eval_script.py
- Zero hallucinations across all queries

Run python eval_script.py --results data/public_results.json to verify yourself.

From 63 million MSEs to every regulated sector in India.

Numbers that matter:

- **3 weeks → 0.8 seconds** = 99.99% time reduction
- **₹15,000 → ₹0** consultant fees per compliance check
- **Offline-capable** — local fallback when NVIDIA API is down

How this scales:

- Building Materials (today)
- FSSAI Food Safety
- AERB Nuclear & Radiation
- MoEF Environmental Compliance
- Every regulated sector in India

Same engine. Swap the PDF. Same accuracy.

Two commands.

```
# 1 — generate results from any test set
python inference.py --input hidden_dataset.json --output team_results.json

# 2 — official evaluation
python eval_script.py --results team_results.json
```

What happens automatically:

- Loads pre-built chunks from data/chunks.json
- Auto-builds ChromaDB embeddings if missing (one-time)
- Pre-warms cross-encoder + embedder
- Runs queries through the full hybrid pipeline
- Filters output through the whitelist
- Writes results in the exact required schema

Tech stack: NVIDIA NIM · ChromaDB · sentence-transformers · rank-bm25 · FastAPI · React 18 · pdfplumber

GitHub: https://github.com/Dushyant-web/bis-rag